¿Cómo funciona la precedencia en un Parser? (La escalera)

En el lenguaje Instinct, una expresión como esta:
health < 20 and enemy_dist < 3

No puede evaluarse de izquierda a derecha sin orden. Debe resolverse respetando la jerarquía de operadores que pide el enunciado:

1. **Primarios: Paréntesis (...), llamadas a función see(x, y), números, strings, variables.**
2. **Unarios: Menos unario -x, operador not.**
3. **Multiplicación/División/Módulo: *, /, %**
4. **Suma/Resta: +, -**
5. **Comparaciones: <, <=, >, >=, ==, !=**
6. **Lógica AND: and**
7. **Lógica OR: or (el de menor prioridad)**

En un Parser por descenso recursivo, esto se implementa como una escalera de métodos: el método de menor precedencia (_parse_or) llama al siguiente (_parse_and), y así sucesivamente hasta llegar a los elementos más básicos (_parse_primary).

Actualización de src/language/parser.py

Vamos a actualizar src/language/parser.py agregando todos los métodos de análisis de expresiones. Reemplaza el contenido completo con este:

Python

```
from typing import List, Optional

from src.language.tokens import Token, TokenType

from src.language.ast_nodes import (

    CompiledProgram,

    Expression,

    LiteralInt,

    LiteralString,

    VariableRef,

    UnaryOp,

    BinaryOp,

    FunctionCall,

)
```

```
from src.language.errors import CompileError


REQUIRED_HEADER_KEYS = {"creature", "faction", "health", "vision", "lifespan"}


class Parser:
    """
    Toma la lista de tokens generada por el Lexer y construye un CompiledProgram.
    Implementa un análisis por descenso recursivo.
    """

    def __init__(self, tokens: List[Token]):
        self.tokens = tokens
        self.current = 0
        self.program = CompiledProgram()

    # ============================================================================
    # PUNTO DE ENTRADA
    # ============================================================================

    def parse(self) -> CompiledProgram:
        """Compila el archivo entero y devuelve un CompiledProgram listo."""
        self._skip_newlines()
        self._parse_header()
        # TANDA 3: self._parse_body()
        return self.program

    # ============================================================================
    # CABECERA (TANDA 1)
```

```
# ============================================================================

def _parse_header(self) -> None:
    seen_keys = set()

    if not self._check_identifier("creature"):
        raise CompileError(
            "La primera línea del archivo debe ser 'creature NombreEspecie'",
            self._peek().line
        )

    while not self._is_at_end():
        if self._is_start_label():
            break

        key_token = self._advance()

        if key_token.type != TokenType.IDENTIFIER:
            raise CompileError(
                f"Se esperaba una clave de cabecera (creature, faction, health, vision, lifespan), "
                f"pero se encontró: '{key_token.value}'",
                key_token.line
            )

        key = key_token.value

        if key not in REQUIRED_HEADER_KEYS:
            raise CompileError(
                f"Clave de cabecera desconocida: '{key}'. "
                f"Se esperaba una de: {REQUIRED_HEADER_KEYS}",
```

```
                key_token.line
            )

        if key in seen_keys:
            raise CompileError(
                f"La clave de cabecera '{key}' está declarada más de una vez",
                key_token.line
            )

        seen_keys.add(key)
        self._parse_header_value(key, key_token.line)
        self._consume_newline(f"Se esperaba salto de línea tras la declaración de '{key}'")
        self._skip_newlines()

    missing = REQUIRED_HEADER_KEYS - seen_keys
    if missing:
        raise CompileError(
            f"Faltan claves obligatorias en la cabecera: {missing}",
            self._peek().line
        )

def _parse_header_value(self, key: str, line: int) -> None:
    value_token = self._advance()

    if key in ("creature", "faction"):
        if value_token.type != TokenType.IDENTIFIER:
            raise CompileError(
                f"'{key}' requiere un nombre (identificador), no '{value_token.value}'",
                line
            )
```

```
        if key == "creature":
            self.program.creature_name = value_token.value
        else:
            self.program.faction = value_token.value

    elif key in ("health", "vision", "lifespan"):
        if value_token.type != TokenType.NUMBER:
            raise CompileError(
                f"'{key}' requiere un número entero, no '{value_token.value}'",
                line
            )
        num = value_token.value

        if key == "health" and num <= 0:
            raise CompileError("'health' debe ser mayor que 0", line)
        if key == "vision" and num < 1:
            raise CompileError("'vision' debe ser mayor o igual que 1", line)
        if key == "lifespan" and num <= 0:
            raise CompileError("'lifespan' debe ser mayor que 0", line)

        if key == "health":
            self.program.health = num
        elif key == "vision":
            self.program.vision = num
        else:
            self.program.lifespan = num

# ============================================================================
# EXPRESIONES (TANDA 2 - Precedencia de menor a mayor)
# ============================================================================
```

```
def parse_expression(self) -> Expression:
    """Punto de entrada para parsear cualquier expresión."""
    return self._parse_or()

def _parse_or(self) -> Expression:
    """Nivel 7: Operador lógico 'or'."""
    expr = self._parse_and()

    while self._match(TokenType.OR):
        right = self._parse_and()
        expr = BinaryOp("or", expr, right)

    return expr

def _parse_and(self) -> Expression:
    """Nivel 6: Operador lógico 'and'."""
    expr = self._parse_comparison()

    while self._match(TokenType.AND):
        right = self._parse_comparison()
        expr = BinaryOp("and", expr, right)

    return expr

def _parse_comparison(self) -> Expression:
    """Nivel 5: Comparaciones (<, <=, >, >=, ==, !=)."""
    expr = self._parse_additive()

    while self._peek().type in (
```

```
            TokenType.LESS,
            TokenType.LESS_EQUAL,
            TokenType.GREATER,
            TokenType.GREATER_EQUAL,
            TokenType.EQUAL_EQUAL,
            TokenType.BANG_EQUAL,
        ):
            op_token = self._advance()
            op_symbols = {
                TokenType.LESS: "<",
                TokenType.LESS_EQUAL: "<=",
                TokenType.GREATER: ">",
                TokenType.GREATER_EQUAL: ">=",
                TokenType.EQUAL_EQUAL: "==",
                TokenType.BANG_EQUAL: "!=",
            }
            right = self._parse_additive()
            expr = BinaryOp(op_symbols[op_token.type], expr, right)

        return expr

    def _parse_additive(self) -> Expression:
        """Nivel 4: Suma (+) y Resta (-)."""
        expr = self._parse_multiplicative()

        while self._peek().type in (TokenType.PLUS, TokenType.MINUS):
            op_token = self._advance()
            op = "+" if op_token.type == TokenType.PLUS else "-"
            right = self._parse_multiplicative()
            expr = BinaryOp(op, expr, right)
```

```
        return expr

    def _parse_multiplicative(self) -> Expression:
        """Nivel 3: Multiplicación (*), División entera (/), Módulo (%)."""
        expr = self._parse_unary()

        while self._peek().type in (TokenType.STAR, TokenType.SLASH, TokenType.PERCENT):
            op_token = self._advance()
            op_map = {
                TokenType.STAR: "*",
                TokenType.SLASH: "/",
                TokenType.PERCENT: "%",
            }
            right = self._parse_unary()
            expr = BinaryOp(op_map[op_token.type], expr, right)

        return expr

    def _parse_unary(self) -> Expression:
        """Nivel 2: Menos unario (-) y Operador 'not'."""
        if self._peek().type in (TokenType.MINUS, TokenType.NOT):
            op_token = self._advance()
            op = "-" if op_token.type == TokenType.MINUS else "not"
            operand = self._parse_unary()  # Permite encadenar: - -5 o not not x
            return UnaryOp(op, operand)

        return self._parse_primary()

    def _parse_primary(self) -> Expression:
```

```
        """Nivel 1: Literales, variables, llamadas a funciones y paréntesis."""
        token = self._peek()

        # 1. Literales Numéricos
        if self._match(TokenType.NUMBER):
            return LiteralInt(token.value)

        # 2. Literales de Texto
        if self._match(TokenType.STRING):
            return LiteralString(token.value)

        # 3. Identificadores (Variables, Percepciones, Constantes) o Llamadas a Funciones
        if self._match(TokenType.IDENTIFIER):
            name = token.value

            # Si le sigue un paréntesis '(', es una llamada a función (ej. see(x, y), name(x, y))
            if self._match(TokenType.LPAREN):
                args = self._parse_argument_list()
                self._consume(TokenType.RPAREN, f"Se esperaba ')' tras los argumentos de la función '{name}'")
                return FunctionCall(name, args)

            # Si no, es una variable, percepción o constante
            return VariableRef(name)

        # 4. Paréntesis agrupadores: ( expresión )
        if self._match(TokenType.LPAREN):
            expr = self.parse_expression()
            self._consume(TokenType.RPAREN, "Se esperaba ')' para cerrar la expresión")
            return expr
```

```
        raise CompileError(
            f"Expresión no válida o inesperada: '{token.value if token.value is not None else token.type.name}'",
            token.line
        )

    def _parse_argument_list(self) -> List[Expression]:
        """Lee una lista de expresiones separadas por comas dentro de paréntesis: arg1, arg2, ..."""
        args: List[Expression] = []

        # Caso: llamada sin argumentos '()'
        if self._check(TokenType.RPAREN):
            return args

        args.append(self.parse_expression())
        while self._match(TokenType.COMMA):
            args.append(self.parse_expression())

        return args

    # ========================================================================
    # UTILIDADES DE NAVEGACIÓN DE TOKENS
    # ========================================================================

    def _peek(self, offset: int = 0) -> Token:
        pos = self.current + offset
        if pos >= len(self.tokens):
            return self.tokens[-1]
        return self.tokens[pos]

    def _advance(self) -> Token:
```

```
    token = self.tokens[self.current]
    if not self._is_at_end():
        self.current += 1
    return token

def _match(self, token_type: TokenType) -> bool:
    """Si el token actual coincide con el tipo, lo consume y devuelve True; si no, False."""
    if self._check(token_type):
        self._advance()
        return True
    return False

def _consume(self, token_type: TokenType, error_msg: str) -> Token:
    """Consume el token actual si coincide con el tipo. Si no, lanza CompileError."""
    if self._check(token_type):
        return self._advance()
    raise CompileError(error_msg, self._peek().line)

def _is_at_end(self) -> bool:
    return self._peek().type == TokenType.EOF

def _check(self, token_type: TokenType) -> bool:
    return self._peek().type == token_type

def _check_identifier(self, name: str) -> bool:
    return self._check(TokenType.IDENTIFIER) and self._peek().value == name

def _is_start_label(self) -> bool:
    return (
        self._check_identifier("start")
```

```
            and self._peek(1).type == TokenType.COLON
        )

    def _skip_newlines(self) -> None:
        while self._check(TokenType.NEWLINE):
            self._advance()

    def _consume_newline(self, error_msg: str) -> None:
        if not self._check(TokenType.NEWLINE) and not self._is_at_end():
            raise CompileError(error_msg, self._peek().line)
        if self._check(TokenType.NEWLINE):
            self._advance()
```

Prueba de Fuego: Comprobar las Expresiones

Vamos a crear o actualizar tests/test_parser.py para probar todas las expresiones del PDF (aritmética compleja, precedencia, funciones see y name, y comparaciones lógicas):

Python

```
from src.language.lexer import Lexer
from src.language.parser import Parser


def parse_single_expr(expr_str: str):
    tokens = Lexer(expr_str).tokenize()
    parser = Parser(tokens)
    return parser.parse_expression()


def test_expressions():
    print("=== PRUEBAS DE EXPRESIONES ===")

    # 1. Precedencia aritmética: 1 + 2 * 3  ->  debe ser 1 + (2 * 3)
```

```
expr1 = parse_single_expr("1 + 2 * 3")
print(f"1 + 2 * 3  -->  {expr1}")

# 2. Expresión del Uruk: random % 3 - 1
expr2 = parse_single_expr("random % 3 - 1")
print(f"random % 3 - 1  -->  {expr2}")

# 3. Expresión del Ent: (home_x > x) - (home_x < x)
expr3 = parse_single_expr("(home_x > x) - (home_x < x)")
print(f"(home_x > x) - (home_x < x)  -->  {expr3}")

# 4. Condición con lógica y comparación: health < 20 and enemy_dist < 3
expr4 = parse_single_expr("health < 20 and enemy_dist < 3")
print(f"health < 20 and enemy_dist < 3  -->  {expr4}")

# 5. Funciones con argumentos complejos: see(x, y + 1) != GROUND
expr5 = parse_single_expr("see(x, y + 1) != GROUND")
print(f"see(x, y + 1) != GROUND  -->  {expr5}")

# 6. Función name con strings: name(x + object_dx, y + object_dy) != \"Rock\"
expr6 = parse_single_expr('name(x + object_dx, y + object_dy) != "Rock"')
print(f'name(...) != "Rock"  -->  {expr6}')

# 7. Operadores unarios combinados: not alive or -enemy_dx
expr7 = parse_single_expr("not alive or -enemy_dx")
print(f"not alive or -enemy_dx  -->  {expr7}")

print("\n✅ ¡Todas las expresiones se parsearon con éxito!")
```

```
if __name__ == "__main__":
    test_expressions()
```